

Classical Machine Learning

**A Napkin-style companion for campus placements:
derivations, hand numerics, traps, and code that checks itself**

Philospher

Built October 6, 2026

Preface

[STUB: preface]

Contents

I	Foundations	9
1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	19
2.3	Probability	19
2.4	MLE and MAP	19
2.5	Convexity	19
2.6	Lagrange multipliers and KKT	19
II	Supervised Learning	21
3	Linear Regression	25
3.1	Least squares and the normal equations	25
3.2	The projection and MLE views	25
3.3	Five points by hand	25
3.4	R-squared and adjusted R-squared	25
3.5	Assumptions, diagnostics, multicollinearity	25
3.6	Polynomial features	25
4	Bias, Variance and Regularisation	27
4.1	The decomposition	27
4.2	Ridge	27
4.3	Lasso and elastic net	27
4.4	The augmented error	27
4.5	Early stopping and double descent	27
5	Logistic Regression and GLMs	29
5.1	From odds to the sigmoid	29
5.2	Log-loss from MLE	29
5.3	Newton and IRLS	29
5.4	Softmax regression and multiclass	29
5.5	Imbalance and thresholds	29
5.6	The perceptron and XOR	29
6	Generative Classifiers	31
6.1	Naive Bayes	31
6.2	Laplace smoothing	31

6.3	LDA and QDA	31
7	k-Nearest Neighbours and Distances	33
7.1	The algorithm	33
7.2	Distances	33
7.3	Choosing k	33
7.4	The curse of dimensionality	33
7.5	KD-trees and approximate search	33
8	Support Vector Machines	35
8.1	Margins	35
8.2	Hard and soft margin	35
8.3	Duality	35
8.4	Kernels	35
8.5	SVR and SMO	35
8.6	Convex surrogate losses	35
9	Decision Trees	37
9.1	Impurity	37
9.2	A split by hand	37
9.3	Growing a tree	37
9.4	Regression trees	37
9.5	Pruning	37
9.6	What trees are good for	37
10	Ensembles	39
10.1	The bootstrap and 63.2 percent	39
10.2	Bagging and random forests	39
10.3	Feature importance and its traps	39
10.4	AdaBoost	39
10.5	Gradient boosting	39
10.6	XGBoost	39
10.7	LightGBM and CatBoost	39
10.8	Stacking and the bias-variance story	39
III	Learning Without (Enough) Labels	41
11	Unsupervised Learning	43
11.1	k-means	43
11.2	Hierarchical clustering	43
11.3	DBSCAN	43
11.4	Gaussian mixtures and EM	43
11.5	PCA	43
11.6	ICA, t-SNE and UMAP	43
11.7	Anomaly detection	43
11.8	Association rules	43
12	Semi- and Self-Supervised Learning	45
12.1	Assumptions	45
12.2	Self-training	45

12.3	Co-training and label propagation	45
12.4	Consistency regularisation	45
12.5	When unlabelled data hurts	45
12.6	Contrastive learning in one page	45
IV	Doing It Properly	47
13	Evaluation and Model Selection	49
13.1	The confusion matrix	49
13.2	ROC and PR curves	49
13.3	Regression and probabilistic metrics	49
13.4	Cross-validation	49
13.5	Leakage	49
13.6	AIC, BIC and hyper-parameter search	49
13.7	Calibration	49
14	Statistics Interviewers Ask	51
14.1	Distributions	51
14.2	Estimators and confidence intervals	51
14.3	Hypothesis tests	51
14.4	Chi-square and ANOVA	51
14.5	Multiple testing	51
14.6	Correlation, causation and Simpson	51
15	Data Preparation and Feature Engineering	53
15.1	Scaling	53
15.2	Encoding categoricals	53
15.3	Missing values and outliers	53
15.4	Feature selection	53
15.5	Imbalance	53
15.6	Text features and TF-IDF	53
15.7	Time series	53
15.8	Recommenders	53
16	Optimisation for ML	55
16.1	Gradient descent on a quadratic	55
16.2	Momentum	55
16.3	Newton and quasi-Newton	55
16.4	Coordinate descent	55
16.5	Duality	55
V	Interview Mode	57
17	Implement It	59
17.1	Linear models	59
17.2	Neighbours and Bayes	59
17.3	Trees and forests	59
17.4	Boosting	59
17.5	Clustering and PCA	59

17.6	SVM	59
18	Rapid-fire	61
18.1	Rapid-fire answers	61
19	Problems	63
19.1	Problems	63
VI	Backmatter	65
A	Hints to problems	67
B	Solutions to problems	69

I

Foundations

Part I: Contents

1	The Setup	11
1.1	The zoo of learning problems	11
1.2	Loss, risk and empirical risk	12
1.3	Train, validation, test	14
1.4	Over- and underfitting, the generalisation gap	14
1.5	The i.i.d. assumption and when it breaks	16
1.6	A few problems to think about	16
2	Maths at Your Fingertips	19
2.1	Linear algebra	19
2.2	Calculus and matrix calculus	19
2.3	Probability	19
2.4	MLE and MAP	19
2.5	Convexity	19
2.6	Lagrange multipliers and KKT	19

1 The Setup

Every interview question in this book is secretly about one sentence: *we choose a function using a finite sample, and we are paid for how it does on data we have not seen*. This chapter fixes the vocabulary for that sentence. It is short on algorithms and long on the distinctions that one-line MCQs love to blur: loss versus risk, validation versus test, training error versus generalisation gap.

§1.1 The zoo of learning problems

Prototypical example for this section: A bank has two million past loan applications, each with a “defaulted / repaid” label, and wants a score for tomorrow’s applicant.

The type of problem is decided by *what supervision you have at training time*, not by the algorithm you later pick.

Definition 1.1.1 (The five regimes). • **Supervised learning**: each training example is a pair $(\mathbf{x}_i, y_i)$, and we learn f with $f(\mathbf{x}) \approx y$. If y is real-valued it is **regression**; if y lies in a finite set it is **classification** (binary or multiclass; **multilabel** if several labels can be on at once).

- **Unsupervised learning**: only $\mathbf{x}_i$. We look for structure: clusters, low-dimensional coordinates, densities, anomalies.
- **Semi-supervised learning**: a few labelled pairs plus many unlabelled $\mathbf{x}$ ’s, and the goal is still a supervised predictor (**Chapter 12**).
- **Self-supervised learning**: labels are *manufactured from the input itself* (mask a word and predict it; crop two views of an image and ask whether they match). It is trained exactly like supervised learning, but nobody paid an annotator.
- **Reinforcement learning**: an agent acts, the environment returns states and a scalar **reward**, and the goal is a policy that maximises long-run reward. There is no correct action given per example, only delayed, noisy feedback on the actions you happened to take.

A useful everyday picture: supervised learning is a student with an answer key, unsupervised is a librarian sorting books with no catalogue, semi-supervised is the librarian who was shown how fifty books were shelved, self-supervised is a student who covers half a page and tries to recall it, and reinforcement learning is learning to cycle — nobody tells you the correct handlebar angle, you just fall less over time.

Task	Regime	Output
Price of a flat from area, floor, locality	supervised	regression
Is this transaction fraud?	supervised	binary classification
Which of 10 digits is in the image?	supervised	multiclass
Which genres does this film belong to?	supervised	multilabel
Segment customers by purchase behaviour	unsupervised	clustering
Flag unusual server logs (no labels)	unsupervised	anomaly score
Pre-train a language model by predicting masked words	self-supervised	classification over vocabulary
Learn to place bids in an ad auction from revenue	reinforcement	policy

Remark 1.1.2 — Trap. “Predicting the number of goals in a match (0, 1, 2, ...) is classification because the output is discrete.” Counts are ordered and arithmetic on them is meaningful, so the usual answer is *regression* (often Poisson regression, [Chapter 5](#)). Discreteness alone does not make a problem classification; *unordered* categories do. Similarly “customer segmentation” is clustering, not classification, unless someone hands you segment labels.

Remark 1.1.3 — Trap. Self-supervised $\neq$ unsupervised in an MCQ. Both use unlabelled data, but self-supervised learning optimises an ordinary supervised loss on a *pretext task*, and its output is usually a representation reused downstream. If an option says “BERT pre-training is unsupervised”, many keys accept it; the precise term is self-supervised.

§1.1.i Parametric versus non-parametric; discriminative versus generative

A **parametric** model has a fixed number of parameters regardless of n (linear and logistic regression). A **non-parametric** model grows with the data (k -nearest neighbours stores all of it; a fully grown tree can have up to n leaves). A **discriminative** model learns $p(y | \mathbf{x})$ or a decision rule directly (logistic regression, SVM); a **generative** model learns $p(\mathbf{x} | y)$ and $p(y)$ and applies Bayes’ rule (naive Bayes, LDA, [Chapter 6](#)).

The regime is set by the supervision available at training time; the output type (real, class, set of classes) then decides regression versus classification.

§1.2 Loss, risk and empirical risk

Prototypical example for this section: Predict $\hat{y} = 2x + 1$ on five points and average the squared errors.

Definition 1.2.1. A **loss** $\ell(\hat{y}, y) \geq 0$ scores one prediction. Common ones: squared loss $(\hat{y} - y)^2$, absolute loss $|\hat{y} - y|$, **0-1 loss** $\mathbb{I}[\hat{y} \neq y]$, log-loss $-\log \hat{p}_y$. Assume data come from an unknown distribution $\mathcal{D}$ over $(\mathbf{x}, y)$. The **risk** (true or expected risk, generalisation error) of f is

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\ell(f(\mathbf{x}), y)],$$

and the **empirical risk** on a sample $S = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is

$$\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i).$$

Empirical risk minimisation (ERM) picks $\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}_S(f)$ over a **hypothesis class** $\mathcal{H}$.

Loss is per example, risk is an expectation over the population, empirical risk is an average over your sample. MCQs swap these words freely; you should not.

Example 1.2.2 (Empirical risk by hand)

Points $x = 0, 1, 2, 3, 4$ with $y = 1.2, 2.9, 5.1, 7.2, 8.7$ and the predictor $f(x) = 2x + 1$. Predictions are 1, 3, 5, 7, 9, so the residuals $y - f(x)$ are

$$+0.2 \quad -0.1 \quad +0.1 \quad +0.2 \quad -0.3 \quad .$$

Mean squared error = 0.038; mean absolute error = 0.180. For a 0-1 example, the rule “predict 1 iff $x > 2$ ” on $x = (0.5, 1.5, 2.2, 2.8, 3.5, 4.1)$, $y = (0, 0, 1, 0, 1, 1)$ makes 1 mistake (the point $x = 2.8$), so the empirical 0-1 risk is $1/6 = 0.1667$.

§1.2.i The best you can possibly do

Even a perfect learner cannot beat the noise in y given $\mathbf{x}$.

Definition 1.2.3. The **Bayes predictor** minimises the risk over *all* functions. Under squared loss it is $f^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$; under 0-1 loss it is $f^*(\mathbf{x}) = \arg \max_c \mathbb{P}(y = c \mid \mathbf{x})$. Its risk is the **Bayes risk** (**Bayes error** for 0-1 loss), the irreducible error.

Why the conditional mean is optimal for squared loss. For fixed $\mathbf{x}$ write $m = \mathbb{E}[y \mid \mathbf{x}]$. Then $\mathbb{E}[(c - y)^2 \mid \mathbf{x}] = (c - m)^2 + \text{Var}(y \mid \mathbf{x})$, minimised at $c = m$; the leftover $\text{Var}(y \mid \mathbf{x})$ is the Bayes risk at $\mathbf{x}$. $\square$

Example 1.2.4 (Bayes error of two Gaussians)

Class 0 is $\mathcal{N}(-1, 1)$, class 1 is $\mathcal{N}(+1, 1)$, equal priors. By symmetry the Bayes rule thresholds at 0, and its error is $\Phi(-1) = 0.1587$; a simulation with 200 000 draws gives 0.1599. No classifier, however deep, does better on this distribution. With priors 0.8/0.2 the optimal threshold moves to $\frac{1}{2} \ln(0.8/0.2) = 0.6931$ (towards the rare class) and the error drops to 0.1121; keeping the threshold at 0 would leave it at 0.1587.

Remark 1.2.5 — Trap. “Bayes error is the error of naive Bayes.” No: Bayes error is a property of the *distribution*, the floor for every classifier. Naive Bayes is one particular (usually sub-optimal) classifier.

§1.2.ii No free lunch, inductive bias

Averaged over *all* possible labelling functions, every learner has the same off-training-set error. Learning works only because real problems are not uniformly random: we assume

smoothness, linearity, sparsity, locality, invariance. These assumptions are the model's **inductive bias**, and choosing $\mathcal{H}$ is choosing one. “Which algorithm is best?” has no answer without a data distribution.

§1.3 Train, validation, test

Prototypical example for this section: Fifty models compared on the same validation set; the winner's score is too good to be true.

Definition 1.3.1. The **training set** fits parameters. The **validation set** (or cross-validation, [Chapter 13](#)) chooses among models and **hyper-parameters** (anything not learnt by the fitting procedure itself: degree, k , λ , tree depth). The **test set** is touched *once*, at the end, to report an unbiased estimate of risk.

Why three sets and not two? Every time you pick the best of several candidates by a score, the winning score is biased upwards: you selected partly on luck. The test set must not have taken part in any choice.

Example 1.3.2 (Selection optimism)

Take 50 “models” that are pure coin flips and score each on a balanced validation set of 100 points. Every model has true accuracy 0.5, yet over 2000 repetitions the *best* validation accuracy averages 0.612 (exact value from the distribution of the maximum of 50 binomials: 0.612). On fresh data the winner scores 0.500. A twelve-point “improvement” produced by selection alone.

Remark 1.3.3 — Typical splits: 60/20/20 or 80/10/10 for moderate data; with millions of rows a 1% test set is already tens of thousands of examples. With little data use k -fold cross-validation for selection and keep a held-out test set if you can afford it; nested CV otherwise ([Chapter 13](#)).

Remark 1.3.4 — Trap. Tuning on the test set (“I tried a few depths and kept the one with best test accuracy”) turns the test set into a validation set. The reported number is then optimistic, exactly as in the coin-flip example. Interviewers probe this with “how many times did you look at the test set?”

§1.4 Over- and underfitting, the generalisation gap

Prototypical example for this section: Fit polynomials of degree 0 to 12 to fifteen noisy samples of $\sin 2\pi x$.

Definition 1.4.1. The **generalisation gap** of $\hat{f}$ is $R(\hat{f}) - \hat{R}_S(\hat{f})$: test (or validation) error minus training error, or equivalently training accuracy minus test accuracy. A model **underfits** when even its training error is high (too little capacity or too much regularisation), and **overfits** when training error is low but the gap is large (it has memorised noise specific to S).

Example 1.4.2 (OA one-liner)

Training accuracy 0.98, test accuracy 0.81. The generalisation gap is 0.17 (17 percentage points): overfitting. Fixes, in order of what interviewers want to hear: more data, regularisation, simpler model / fewer features, early stopping, ensembling (bagging), data augmentation.

Example 1.4.3 (The degree sweep)

Fifteen training points, $y = \sin(2\pi x) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.3^2)$ (so the Bayes risk is 0.09). Median MSE over 200 repetitions:

degree	train MSE	test MSE	gap
0	0.558	0.611	0.053
1	0.238	0.323	0.085
3	0.065	0.124	0.059
9	0.037	0.777	0.740

Degree 0 and 1 underfit (both errors high, gap small); degree 3 is best on test; degree 9 has the lowest training error and a huge gap. The *mean* test MSE at degree 9 is 3768, because a few unlucky fits explode between and beyond the training points — variance in its purest form.

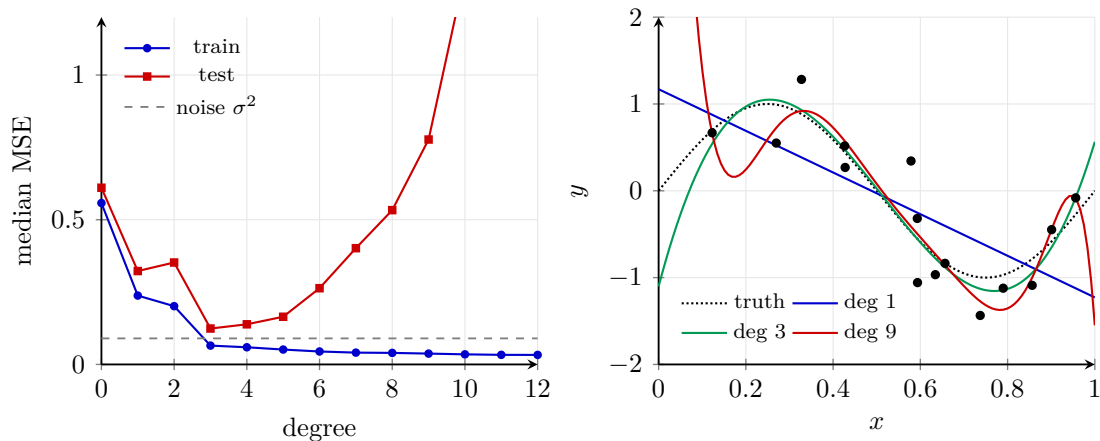


Figure 1.1: Left: training error falls monotonically with capacity; test error is U-shaped. Right: degree 1 underfits, degree 3 is about right, degree 9 threads every noisy point.

Remark 1.4.4 — Trap. “A small generalisation gap means a good model.” A constant predictor has a tiny gap and is useless. You need *both* low training error and a small gap. Conversely a modest gap with the best validation score is fine; deep nets routinely have training error near zero and are still the best model available.

Remark 1.4.5 — Trap. “Training error keeps decreasing, so we are still improving.” Training error is non-increasing in capacity for nested model classes almost by definition; it says nothing about generalisation. Watch validation error.

Question 1.4.6. If you add an irrelevant random feature to a least-squares model, can its training MSE go up? (Answer: no — the old solution is still available with weight 0 on the new feature.)

§1.5 The i.i.d. assumption and when it breaks

Prototypical example for this section: A model trained on 2022 transactions is scored on 2024 transactions after a payment app changes its UI.

Everything above assumes the training points and future points are drawn **independently and identically distributed (i.i.d.)** from the same $\mathcal{D}$. The two halves fail differently.

- **Not independent.** Several rows per patient, per user, per session; consecutive time steps. A random row-level split puts the same patient in train and test, and the model scores well by recognising the patient. Fix: split by group (GroupKFold) or by time.
- **Not identically distributed (distribution shift).** **Covariate shift:** $p(\mathbf{x})$ changes, $p(y | \mathbf{x})$ does not (a new city with different house sizes). **Label shift:** $p(y)$ changes, $p(\mathbf{x} | y)$ does not (fraud rate doubles during a festival sale). **Concept drift:** $p(y | \mathbf{x})$ itself changes (what counts as spam evolves).

Remark 1.5.1 — Trap. Shuffling a time series before a train/test split leaks the future into the past: the model sees Tuesday when predicting Monday. Use forward-chaining splits (Chapter 13).

Training error measures fit; the gap measures luck you mistook for signal; the i.i.d. assumption is what lets either number say anything about tomorrow.

§1.6 A few problems to think about

Problem 1A (Name the regime). Classify each as supervised (regression/classification), unsupervised, self-supervised or RL: (a) predicting tomorrow’s maximum temperature; (b) grouping news articles into topics with no labels; (c) predicting the next frame of a video from previous frames; (d) learning to play Ludo from win/loss outcomes; (e) assigning one of five star ratings to a review.

hint · solution

Problem 1B (Empirical risk). A predictor outputs $\hat{y} = (3, 5, 2, 8)$ for targets $y = (2, 5, 4, 7)$. Compute the mean squared error and the mean absolute error. Which of the two is more affected if the last target were 17 instead of 7?

hint · solution

Problem 1C (Gap arithmetic). Model A: train accuracy 0.99, validation 0.84. Model B: train 0.88, validation 0.86. Model C: train 0.71, validation 0.70. Which overfits, which underfits, which would you ship?

[hint](#) · [solution](#)

Problem 1D (Bayes rule with priors). Class 0 is $\mathcal{N}(0, 1)$ and class 1 is $\mathcal{N}(2, 1)$, with $\mathbb{P}(y = 1) = \pi$. Show that the Bayes classifier predicts 1 iff $x > 1 + \frac{1}{2} \ln \frac{1-\pi}{\pi}$. What happens as $\pi \rightarrow 0$?

[hint](#) · [solution](#)

Problem 1E (Leaky split). A hospital dataset has 10 X-rays each for 500 patients; the label is a diagnosis per patient. A random 80/20 split of the 5000 images gives 97% test accuracy, but the deployed model gets 78%. Explain, and propose a correct validation scheme.

[hint](#) · [solution](#)

Problem 1F (Selection bias, quantified). You evaluate m useless classifiers (each predicts with a fair coin) on a balanced validation set of size n . Using the normal approximation, estimate the expected best validation accuracy for $m = 50$, $n = 100$. (Use $\mathbb{E}[\max \text{ of } 50 \text{ standard normals}] \approx 2.25$.)

[hint](#) · [solution](#)

2 Maths at Your Fingertips

§2.1 Linear algebra

[STUB: Linear algebra]

§2.2 Calculus and matrix calculus

[STUB: Calculus and matrix calculus]

§2.3 Probability

[STUB: Probability]

§2.4 MLE and MAP

[STUB: MLE and MAP]

§2.5 Convexity

[STUB: Convexity]

§2.6 Lagrange multipliers and KKT

[STUB: Lagrange multipliers and KKT]

II

Supervised Learning

Part II: Contents

3	Linear Regression	25
3.1	Least squares and the normal equations	25
3.2	The projection and MLE views	25
3.3	Five points by hand	25
3.4	R-squared and adjusted R-squared	25
3.5	Assumptions, diagnostics, multicollinearity	25
3.6	Polynomial features	25
4	Bias, Variance and Regularisation	27
4.1	The decomposition	27
4.2	Ridge	27
4.3	Lasso and elastic net	27
4.4	The augmented error	27
4.5	Early stopping and double descent	27
5	Logistic Regression and GLMs	29
5.1	From odds to the sigmoid	29
5.2	Log-loss from MLE	29
5.3	Newton and IRLS	29
5.4	Softmax regression and multiclass	29
5.5	Imbalance and thresholds	29
5.6	The perceptron and XOR	29
6	Generative Classifiers	31
6.1	Naive Bayes	31
6.2	Laplace smoothing	31
6.3	LDA and QDA	31
7	k-Nearest Neighbours and Distances	33
7.1	The algorithm	33
7.2	Distances	33
7.3	Choosing k	33
7.4	The curse of dimensionality	33
7.5	KD-trees and approximate search	33
8	Support Vector Machines	35
8.1	Margins	35
8.2	Hard and soft margin	35
8.3	Duality	35
8.4	Kernels	35
8.5	SVR and SMO	35
8.6	Convex surrogate losses	35
9	Decision Trees	37
9.1	Impurity	37
9.2	A split by hand	37
9.3	Growing a tree	37
9.4	Regression trees	37
9.5	Pruning	37
9.6	What trees are good for	37
10	Ensembles	39
10.1	The bootstrap and 63.2 percent	39
10.2	Bagging and random forests	39
10.3	Feature importance and its traps	39

10.4	AdaBoost	39
10.5	Gradient boosting	39
10.6	XGBoost	39
10.7	LightGBM and CatBoost	39
10.8	Stacking and the bias-variance story	39

3 Linear Regression

§3.1 Least squares and the normal equations

[STUB: Least squares and the normal equations]

§3.2 The projection and MLE views

[STUB: The projection and MLE views]

§3.3 Five points by hand

[STUB: Five points by hand]

§3.4 R-squared and adjusted R-squared

[STUB: R-squared and adjusted R-squared]

§3.5 Assumptions, diagnostics, multicollinearity

[STUB: Assumptions, diagnostics, multicollinearity]

§3.6 Polynomial features

[STUB: Polynomial features]

4 Bias, Variance and Regularisation

§4.1 The decomposition

[STUB: The decomposition]

§4.2 Ridge

[STUB: Ridge]

§4.3 Lasso and elastic net

[STUB: Lasso and elastic net]

§4.4 The augmented error

[STUB: The augmented error]

§4.5 Early stopping and double descent

[STUB: Early stopping and double descent]

5 Logistic Regression and GLMs

§5.1 From odds to the sigmoid

[STUB: From odds to the sigmoid]

§5.2 Log-loss from MLE

[STUB: Log-loss from MLE]

§5.3 Newton and IRLS

[STUB: Newton and IRLS]

§5.4 Softmax regression and multiclass

[STUB: Softmax regression and multiclass]

§5.5 Imbalance and thresholds

[STUB: Imbalance and thresholds]

§5.6 The perceptron and XOR

[STUB: The perceptron and XOR]

6 Generative Classifiers

§6.1 Naive Bayes

[STUB: Naive Bayes]

§6.2 Laplace smoothing

[STUB: Laplace smoothing]

§6.3 LDA and QDA

[STUB: LDA and QDA]

7 k-Nearest Neighbours and Distances

§7.1 The algorithm

[STUB: The algorithm]

§7.2 Distances

[STUB: Distances]

§7.3 Choosing k

[STUB: Choosing k]

§7.4 The curse of dimensionality

[STUB: The curse of dimensionality]

§7.5 KD-trees and approximate search

[STUB: KD-trees and approximate search]

8 Support Vector Machines

§8.1 Margins

[STUB: Margins]

§8.2 Hard and soft margin

[STUB: Hard and soft margin]

§8.3 Duality

[STUB: Duality]

§8.4 Kernels

[STUB: Kernels]

§8.5 SVR and SMO

[STUB: SVR and SMO]

§8.6 Convex surrogate losses

[STUB: Convex surrogate losses]

9 Decision Trees

§9.1 Impurity

[STUB: Impurity]

§9.2 A split by hand

[STUB: A split by hand]

§9.3 Growing a tree

[STUB: Growing a tree]

§9.4 Regression trees

[STUB: Regression trees]

§9.5 Pruning

[STUB: Pruning]

§9.6 What trees are good for

[STUB: What trees are good for]

10 Ensembles

§10.1 The bootstrap and 63.2 percent

[STUB: The bootstrap and 63.2 percent]

§10.2 Bagging and random forests

[STUB: Bagging and random forests]

§10.3 Feature importance and its traps

[STUB: Feature importance and its traps]

§10.4 AdaBoost

[STUB: AdaBoost]

§10.5 Gradient boosting

[STUB: Gradient boosting]

§10.6 XGBoost

[STUB: XGBoost]

§10.7 LightGBM and CatBoost

[STUB: LightGBM and CatBoost]

§10.8 Stacking and the bias-variance story

[STUB: Stacking and the bias-variance story]

III

Learning Without (Enough) Labels

Part III: Contents

11	Unsupervised Learning	43
11.1	k-means	43
11.2	Hierarchical clustering	43
11.3	DBSCAN	43
11.4	Gaussian mixtures and EM	43
11.5	PCA	43
11.6	ICA, t-SNE and UMAP	43
11.7	Anomaly detection	43
11.8	Association rules	43
12	Semi- and Self-Supervised Learning	45
12.1	Assumptions	45
12.2	Self-training	45
12.3	Co-training and label propagation	45
12.4	Consistency regularisation	45
12.5	When unlabelled data hurts	45
12.6	Contrastive learning in one page	45

11 Unsupervised Learning

§11.1 k-means

[STUB: k-means]

§11.2 Hierarchical clustering

[STUB: Hierarchical clustering]

§11.3 DBSCAN

[STUB: DBSCAN]

§11.4 Gaussian mixtures and EM

[STUB: Gaussian mixtures and EM]

§11.5 PCA

[STUB: PCA]

§11.6 ICA, t-SNE and UMAP

[STUB: ICA, t-SNE and UMAP]

§11.7 Anomaly detection

[STUB: Anomaly detection]

§11.8 Association rules

[STUB: Association rules]

12 Semi- and Self-Supervised Learning

§12.1 Assumptions

[STUB: Assumptions]

§12.2 Self-training

[STUB: Self-training]

§12.3 Co-training and label propagation

[STUB: Co-training and label propagation]

§12.4 Consistency regularisation

[STUB: Consistency regularisation]

§12.5 When unlabelled data hurts

[STUB: When unlabelled data hurts]

§12.6 Contrastive learning in one page

[STUB: Contrastive learning in one page]

IV

Doing It Properly

Part IV: Contents

13	Evaluation and Model Selection	49
13.1	The confusion matrix	49
13.2	ROC and PR curves	49
13.3	Regression and probabilistic metrics	49
13.4	Cross-validation	49
13.5	Leakage	49
13.6	AIC, BIC and hyper-parameter search	49
13.7	Calibration	49
14	Statistics Interviewers Ask	51
14.1	Distributions	51
14.2	Estimators and confidence intervals	51
14.3	Hypothesis tests	51
14.4	Chi-square and ANOVA	51
14.5	Multiple testing	51
14.6	Correlation, causation and Simpson	51
15	Data Preparation and Feature Engineering	53
15.1	Scaling	53
15.2	Encoding categoricals	53
15.3	Missing values and outliers	53
15.4	Feature selection	53
15.5	Imbalance	53
15.6	Text features and TF-IDF	53
15.7	Time series	53
15.8	Recommenders	53
16	Optimisation for ML	55
16.1	Gradient descent on a quadratic	55
16.2	Momentum	55
16.3	Newton and quasi-Newton	55
16.4	Coordinate descent	55
16.5	Duality	55

13 Evaluation and Model Selection

§13.1 The confusion matrix

[STUB: The confusion matrix]

§13.2 ROC and PR curves

[STUB: ROC and PR curves]

§13.3 Regression and probabilistic metrics

[STUB: Regression and probabilistic metrics]

§13.4 Cross-validation

[STUB: Cross-validation]

§13.5 Leakage

[STUB: Leakage]

§13.6 AIC, BIC and hyper-parameter search

[STUB: AIC, BIC and hyper-parameter search]

§13.7 Calibration

[STUB: Calibration]

14 Statistics Interviewers Ask

§14.1 Distributions

[STUB: Distributions]

§14.2 Estimators and confidence intervals

[STUB: Estimators and confidence intervals]

§14.3 Hypothesis tests

[STUB: Hypothesis tests]

§14.4 Chi-square and ANOVA

[STUB: Chi-square and ANOVA]

§14.5 Multiple testing

[STUB: Multiple testing]

§14.6 Correlation, causation and Simpson

[STUB: Correlation, causation and Simpson]

15 Data Preparation and Feature Engineering

§15.1 Scaling

[STUB: Scaling]

§15.2 Encoding categoricals

[STUB: Encoding categoricals]

§15.3 Missing values and outliers

[STUB: Missing values and outliers]

§15.4 Feature selection

[STUB: Feature selection]

§15.5 Imbalance

[STUB: Imbalance]

§15.6 Text features and TF-IDF

[STUB: Text features and TF-IDF]

§15.7 Time series

[STUB: Time series]

§15.8 Recommenders

[STUB: Recommenders]

16 Optimisation for ML

§16.1 Gradient descent on a quadratic

[STUB: Gradient descent on a quadratic]

§16.2 Momentum

[STUB: Momentum]

§16.3 Newton and quasi-Newton

[STUB: Newton and quasi-Newton]

§16.4 Coordinate descent

[STUB: Coordinate descent]

§16.5 Duality

[STUB: Duality]



Interview Mode

Part V: Contents

17	Implement It	59
17.1	Linear models	59
17.2	Neighbours and Bayes	59
17.3	Trees and forests	59
17.4	Boosting	59
17.5	Clustering and PCA	59
17.6	SVM	59
18	Rapid-fire	61
18.1	Rapid-fire answers	61
19	Problems	63
19.1	Problems	63

17 Implement It

§17.1 Linear models

[STUB: Linear models]

§17.2 Neighbours and Bayes

[STUB: Neighbours and Bayes]

§17.3 Trees and forests

[STUB: Trees and forests]

§17.4 Boosting

[STUB: Boosting]

§17.5 Clustering and PCA

[STUB: Clustering and PCA]

§17.6 SVM

[STUB: SVM]

18 Rapid-fire

§18.1 Rapid-fire answers

[STUB: Rapid-fire answers]

19 Problems

§19.1 Problems

[STUB: Problems]

VI

Backmatter

A Hints to problems

- 1A.** Ask what plays the role of y at training time, and where it came from.
- 1B.** Residuals are $(1, 0, -2, 1)$.
- 1C.** Look at both the level and the gap.
- 1D.** Compare $\pi \phi(x - 2)$ with $(1 - \pi) \phi(x)$ and take logs.
- 1E.** Are the rows independent?
- 1F.** Each accuracy is approximately $\mathcal{N}(0.5, 0.25/n)$.

B Solutions to problems

- 1A.** (a) supervised regression. (b) unsupervised clustering/topic modelling. (c) self-supervised: the target frame is part of the input stream. (d) reinforcement learning: only delayed reward. (e) supervised; strictly ordinal regression, but usually treated as five-class classification — in an interview say that the order matters and that a model ignoring it penalises predicting 4 for a 5 as much as predicting 1.
- 1B.** $\text{MSE} = (1 + 0 + 4 + 1)/4 = 1.5$; $\text{MAE} = (1 + 0 + 2 + 1)/4 = 1$. With target 17 the last residual is -9 : MSE becomes $(1 + 0 + 4 + 81)/4 = 21.5$, MAE becomes $(1 + 0 + 2 + 9)/4 = 3$. Squared loss is far more sensitive to outliers, which is why MAE or Huber loss is used when targets have heavy tails.
- 1C.** A overfits (gap 0.15). C underfits (both low, gap 0.01). B has the best validation accuracy and a small gap; ship B (after confirming on the untouched test set). Note that A has the best training accuracy, and that the overfitting A still beats the underfitting C on validation (0.84 versus 0.70): overfitting is not automatically worse than underfitting; compare validation scores.
- 1D.** Predict 1 iff $\pi e^{-(x-2)^2/2} > (1-\pi)e^{-x^2/2}$, i.e. $\ln \pi - (x-2)^2/2 > \ln(1-\pi) - x^2/2$, i.e. $2x - 2 > \ln \frac{1-\pi}{\pi}$, which is the claim. As $\pi \rightarrow 0$ the threshold goes to $+\infty$: when positives are rare you need overwhelming evidence to predict one, which is why accuracy-optimal classifiers on imbalanced data predict “negative” almost always ([Chapter 13](#)).
- 1E.** Images of the same patient land in both train and test; the network can recognise the patient (anatomy, implants, scanner artefacts) and copy that patient’s label. The test score measures memorisation, not diagnosis. Split by patient (`GroupKFold` with patient ID as the group), and ideally also by hospital/scanner if deployment will see new sites.
- 1F.** Standard deviation $= \sqrt{0.25/100} = 0.05$, so the best is about $0.5 + 2.25 \times 0.05 = 0.6125$, matching the simulation 0.612 in the chapter. The optimism shrinks like $1/\sqrt{n}$ and grows like $\sqrt{2 \ln m}$: bigger validation sets help, comparing more models hurts.